Lab 04 — Microservicios con JWT, OpenFeign, filtro de seguridad y Docker

Integrantes

Jhonnatan David Hernandez Martinez - 2201492
Diego Alejandro Gómez Pedraza - 2211060
Luis Sebastián Mora Cañas - 2211554

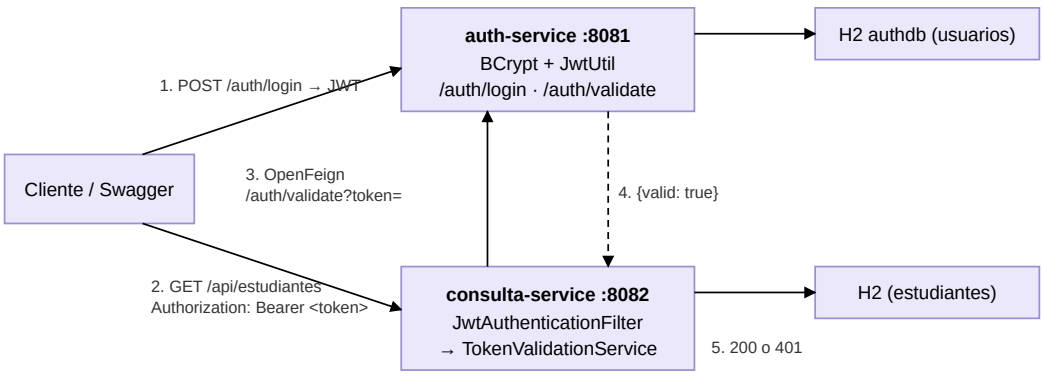
Se construyó el proyecto base `demo-academico` desde cero y a partir de él los microservicios `consulta-service` y `auth-service`, integrados con OpenFeign y desplegados con Docker Compose. El código está en la carpeta `microservicios-demoacademico/`.

Actividades

1. **Parte 0:** proyecto base con la arquitectura controller → handler → service → repository → model, DTOs de creación, actualización y salida, `ApiResponse`/`ResponseBuilder`, `GlobalExceptionHandler` y un seeder de 500 estudiantes.
2. **Parte 1:** se copió el base como `consulta-service` (puerto 8082) y se agregaron OpenFeign, Spring Security y `@EnableFeignClients`.
3. **Parte 2:** `auth-service` (puerto 8081) con `AuthUser` en H2, contraseñas cifradas con BCrypt, seeder de 3 usuarios, `JwtUtil` (HS256, 1 hora de vigencia) y los endpoints `/auth/login` y `/auth/validate`.
4. **Parte 3:** `AuthFeignClient` y `TokenValidationService` en consulta-service, que preguntan a auth-service si un token es válido.
5. **Parte 4:** `OpenApiConfig` con el esquema Bearer (botón Authorize), `JwtAuthenticationFilter`, que intercepta `/api/estudiantes`, y `SecurityConfig`.
6. **Parte 5:** un Dockerfile por servicio y un `docker-compose.yml` con la red `micro-net`. Dentro de Docker, consulta-service llama a `http://auth-service:8081`.
7. Se ejecutó el guion obligatorio en Swagger y se escribió un README con la ejecución, las URLs, los usuarios y el uso de Authorize.

Corrección necesaria: el proyecto base, tal como está en la guía, compila pero no arranca: `javafaker 1.0.2` trae SnakeYAML 1.23, que choca con Spring Boot (`NoSuchMethodError: LoaderOptions.setMaxAliasesForCollections`). Se excluyó `org.yaml:snakeyaml` de javafaker en el `pom.xml`; el seeder siguió funcionando y consulta-service heredó la corrección.

Diagrama



Verificación del entorno

```
openjdk version "21.0.12.1" 2026-08-18
OpenJDK Runtime Environment (build 21.0.12.1+1)
```

```
OpenJDK 64-Bit Server VM (build 21.0.12.1+1, mixed mode, sharing)
Apache Maven 3.10.0 (c43a36b8d67be7e0805a411bc0898af1a51f5472)
Docker version 28.3.3, build 980b856
```

Parte 0 — Proyecto base: compilación y CRUD

```
$ mvn clean package -DskipTests
[INFO] Building jar: /home/jhonatan/Documents/microservicios-demoacademico/demo-academico/target/demo-
academico-0.0.1-SNAPSHOT.jar
[INFO] BUILD SUCCESS
[INFO] Total time:  4.402 s
```

```
1. Listar (seeder de 500)
GET http://localhost:8080/api/estudiantes?page=0&size=1
→ HTTP 200 ✔
{"success": true, "message": "OK", "data": {"totalElements": 500, "content[0]": {"id": 1, "nombre":
"Jesús", "apellido": "Manzanares", "email": "estudiante0@demoacademico.edu", "edad": 18}}}

2. Crear estudiante
POST http://localhost:8080/api/estudiantes
{"nombre": "Ana", "apellido": "Torres", "email": "ana.torres@uis.edu.co", "edad": 21}
→ HTTP 201 ✔
{"success": true, "message": "Estudiante creado", "data": {"id": 501, "nombre": "Ana", "apellido":
"Torres", "email": "ana.torres@uis.edu.co", "edad": 21}}

3. Obtener por id
GET http://localhost:8080/api/estudiantes/501
→ HTTP 200 ✔
{"success": true, "message": "OK", "data": {"id": 501, "nombre": "Ana", "apellido": "Torres", "email":
"ana.torres@uis.edu.co", "edad": 21}}

4. Actualizar
PUT http://localhost:8080/api/estudiantes/501
{"nombre": "Ana María", "apellido": "Torres", "email": "ana.torres@uis.edu.co", "edad": 22}
→ HTTP 200 ✔
{"success": true, "message": "Estudiante actualizado", "data": {"id": 501, "nombre": "Ana María",
"apellido": "Torres", "email": "ana.torres@uis.edu.co", "edad": 22}}

5. Email duplicado (BusinessException)
POST http://localhost:8080/api/estudiantes
{"nombre": "Otro", "apellido": "X", "email": "ana.torres@uis.edu.co", "edad": 20}
→ HTTP 400 ✔
{"success": false, "message": "Ya existe un estudiante con el email: ana.torres@uis.edu.co", "data": null}

6. Validación
POST http://localhost:8080/api/estudiantes
{"nombre": "", "apellido": "X", "email": "no-es-email", "edad": 0}
→ HTTP 400 ✔
{"success": false, "message": "Error de validación", "data": {"email": "El email no es válido", "nombre":
"El nombre es obligatorio", "edad": "La edad mínima es 1"}}

7. Eliminar
DELETE http://localhost:8080/api/estudiantes/501
→ HTTP 200 ✔
{"success": true, "message": "Estudiante eliminado", "data": null}

8. Obtener eliminado (NotFoundException)
GET http://localhost:8080/api/estudiantes/501
→ HTTP 404 ✔
{"success": false, "message": "Estudiante no encontrado: 501", "data": null}

RESULTADO: 8/8 pasos con el status esperado
```

RESULTADO: 8/8 pasos con el status esperado.

Partes 1 a 4 — Compilación de ambos servicios

```
$ cd auth-service && mvn clean package -DskipTests
[INFO] Building jar: /home/jhonatan/Documents/microservicios-demoacademico/auth-service/target/auth-
service-0.0.1-SNAPSHOT.jar
```

```
[INFO] BUILD SUCCESS
[INFO] Total time:  5.809 s
```

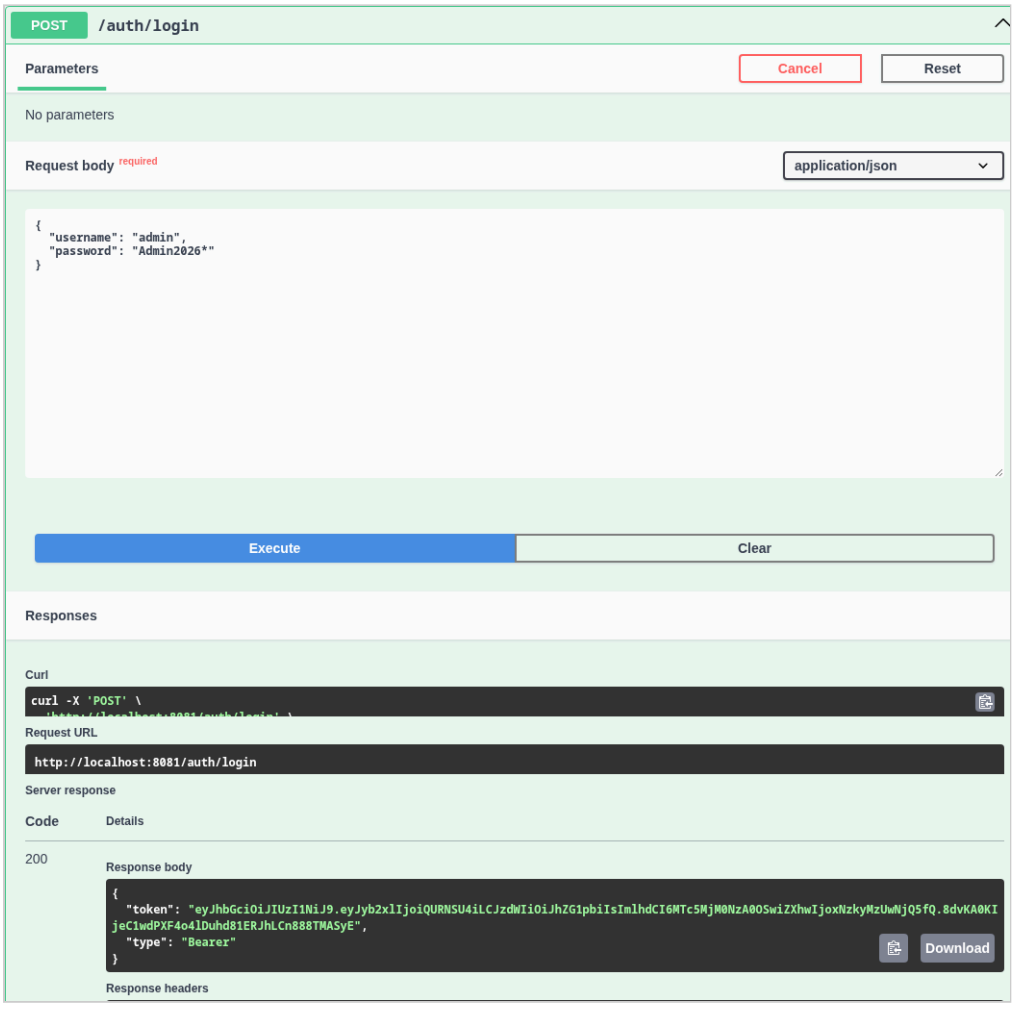
```
$ cd consulta-service && mvn clean package -DskipTests
[INFO] Building jar: /home/jhonatan/Documents/microservicios-demoacademico/consulta-
service/target/consulta-service-0.0.1-SNAPSHOT.jar
[INFO] BUILD SUCCESS
[INFO] Total time:  4.875 s
```

Parte 5 — Docker Compose

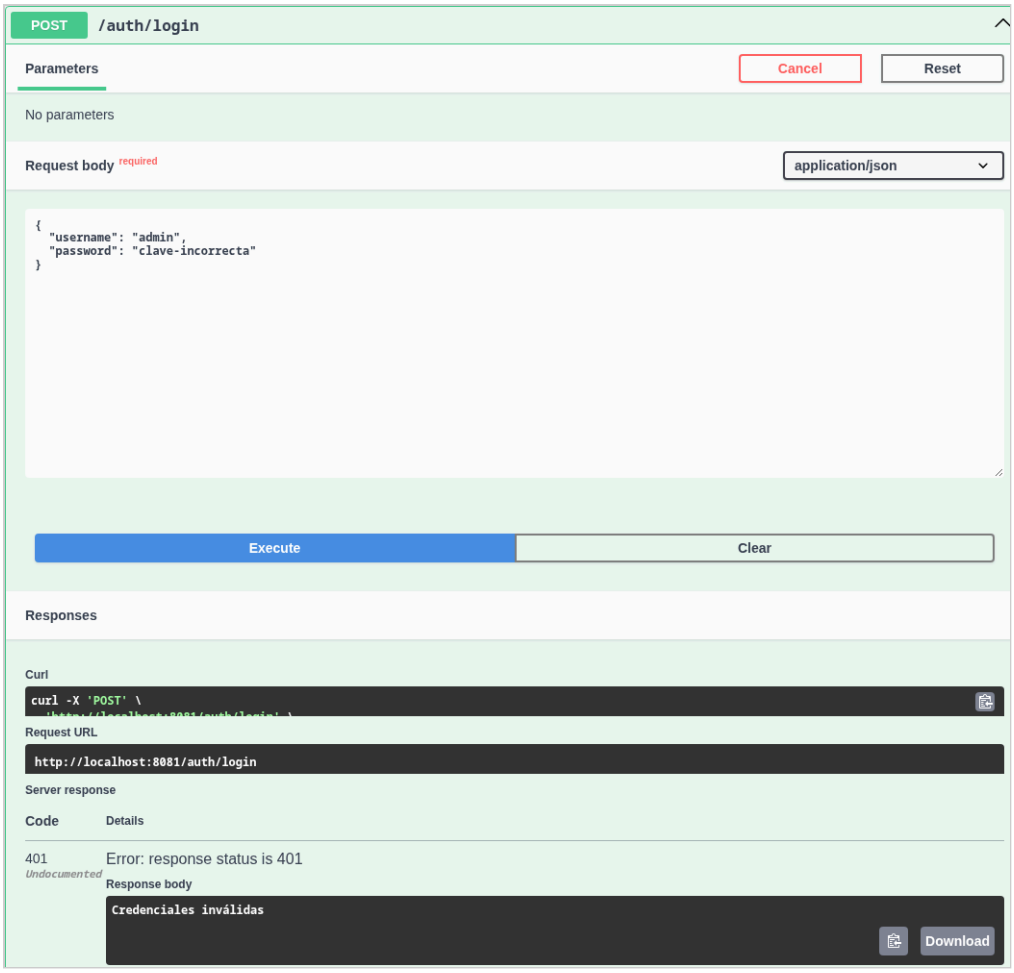
```
$ docker compose up --build -d
…
 Network microservicios-demoacademico_micro-net Created
 Network microservicios-demoacademico_micro-net Created
 Container auth-service Created
 Container consulta-service Created
 Container auth-service Started
 Container consulta-service Started
```

```
$ docker ps
NAMES              IMAGE                                          STATUS          PORTS
consulta-service   microservicios-demoacademico-consulta-service   Up 10 seconds   0.0.0.0:8082->8082/tcp,
[::]:8082->8082/tcp
auth-service       microservicios-demoacademico-auth-service       Up 10 seconds   0.0.0.0:8081->8081/tcp,
[::]:8081->8081/tcp
```

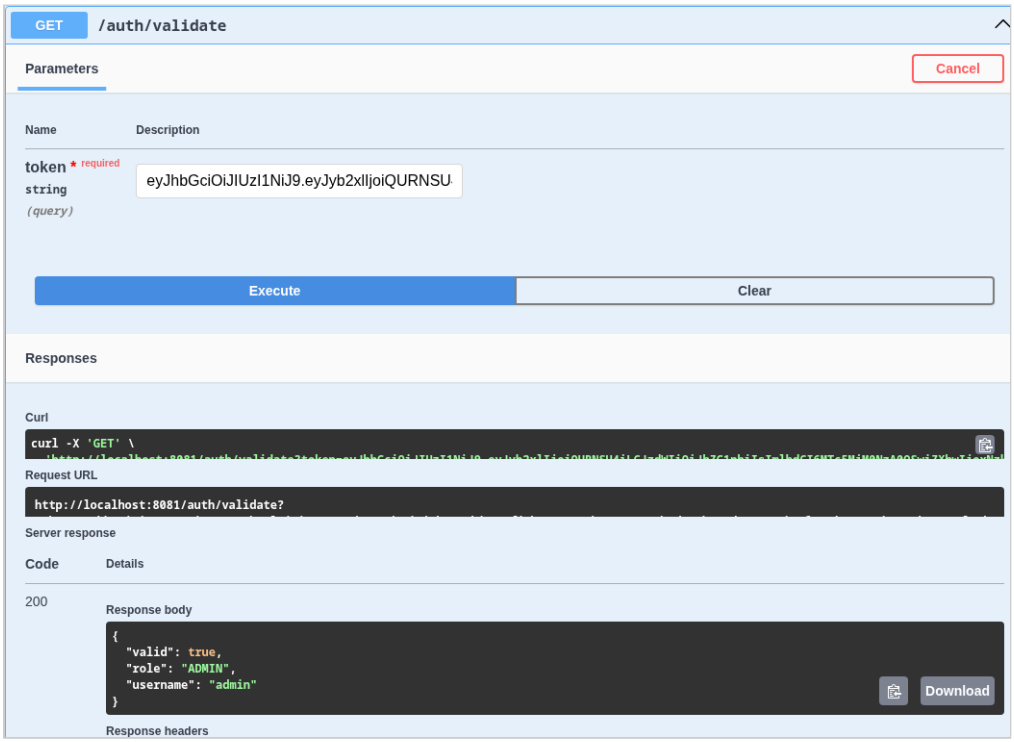
Prueba 1 — Login exitoso (admin)



Prueba 2 — Login fallido con contraseña incorrecta

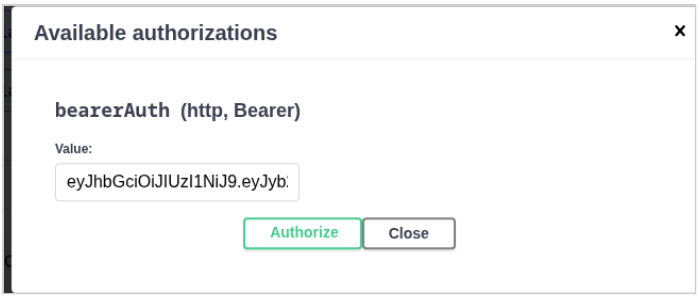


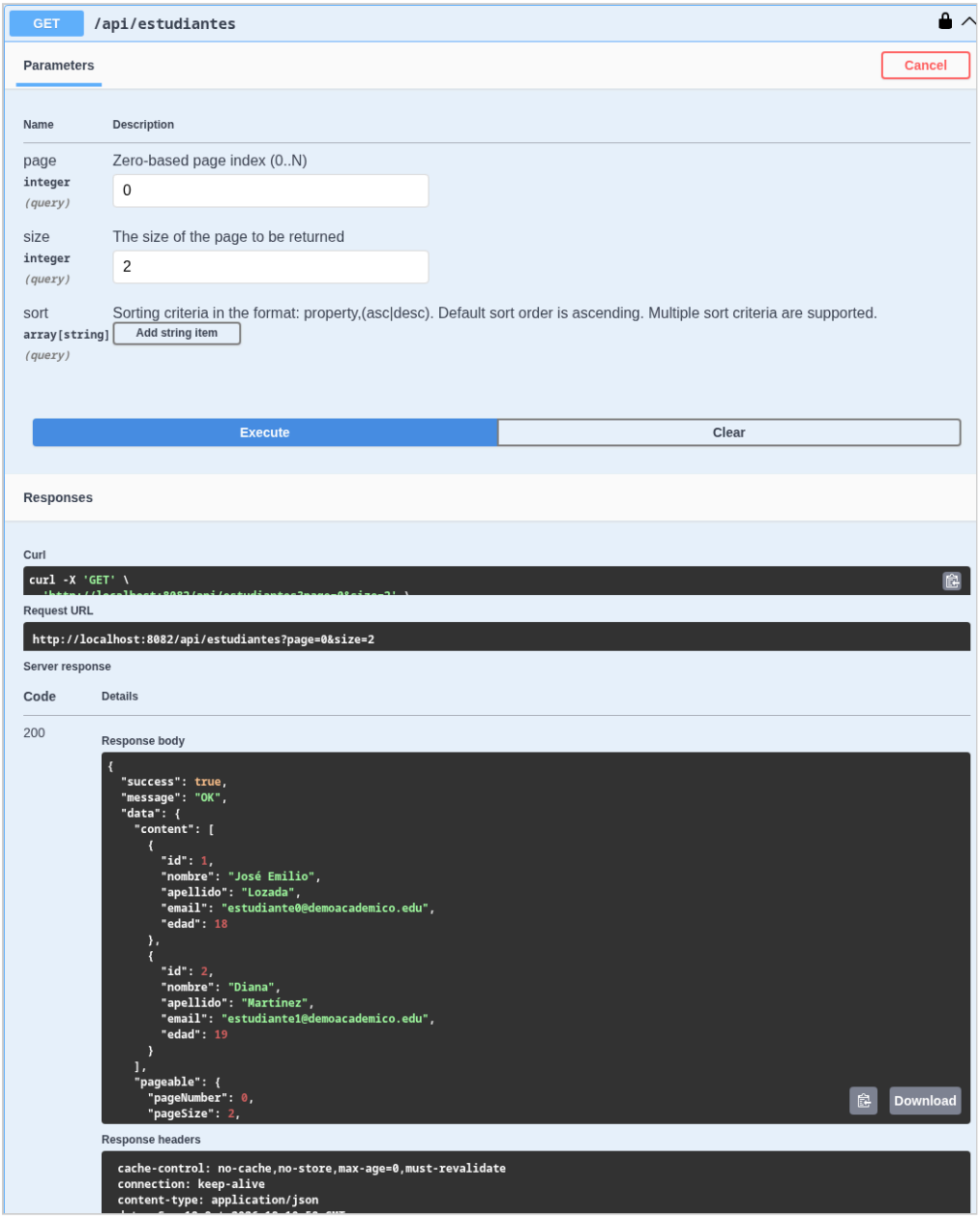
Prueba 3 — Validación manual del token



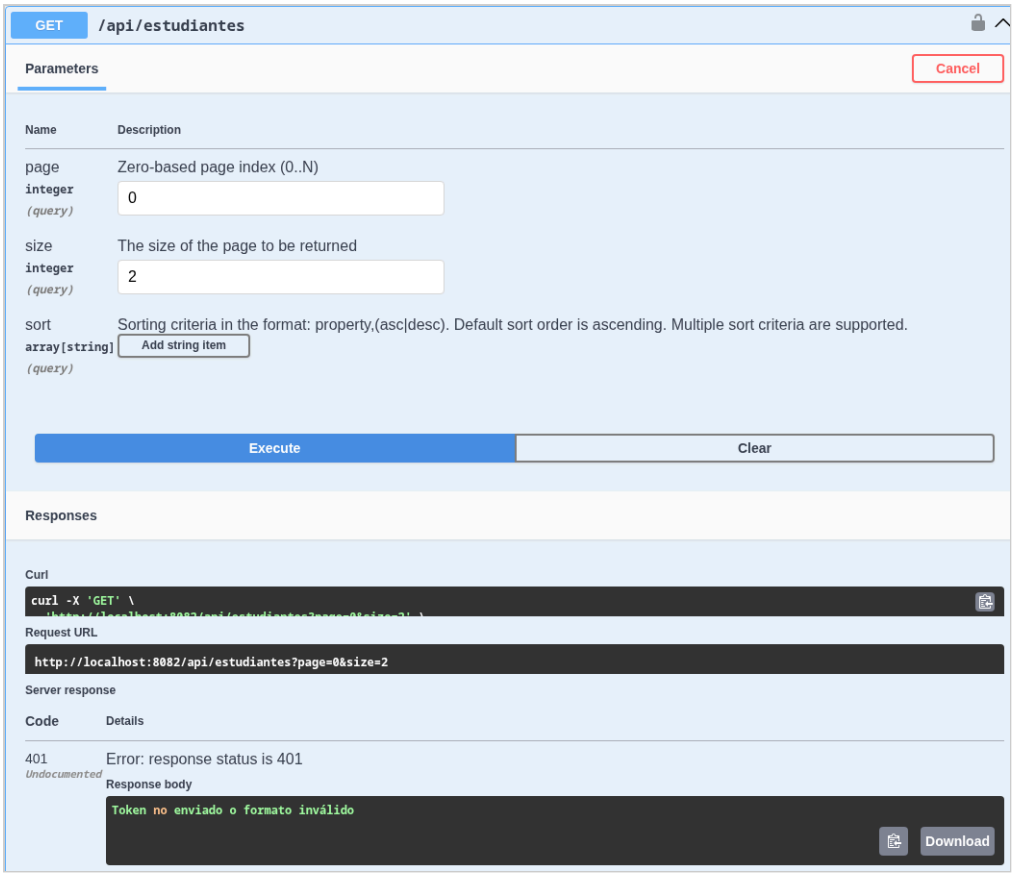
Prueba 4 — GET /api/estudiantes con token válido

Se pegó solo el token en Authorize, sin la palabra Bearer:


Available authorizations
bearerAuth (http, Bearer)
Value:
eyJhbGciOiJIUzI1NiJ9.eyJyb
Authorize
Close

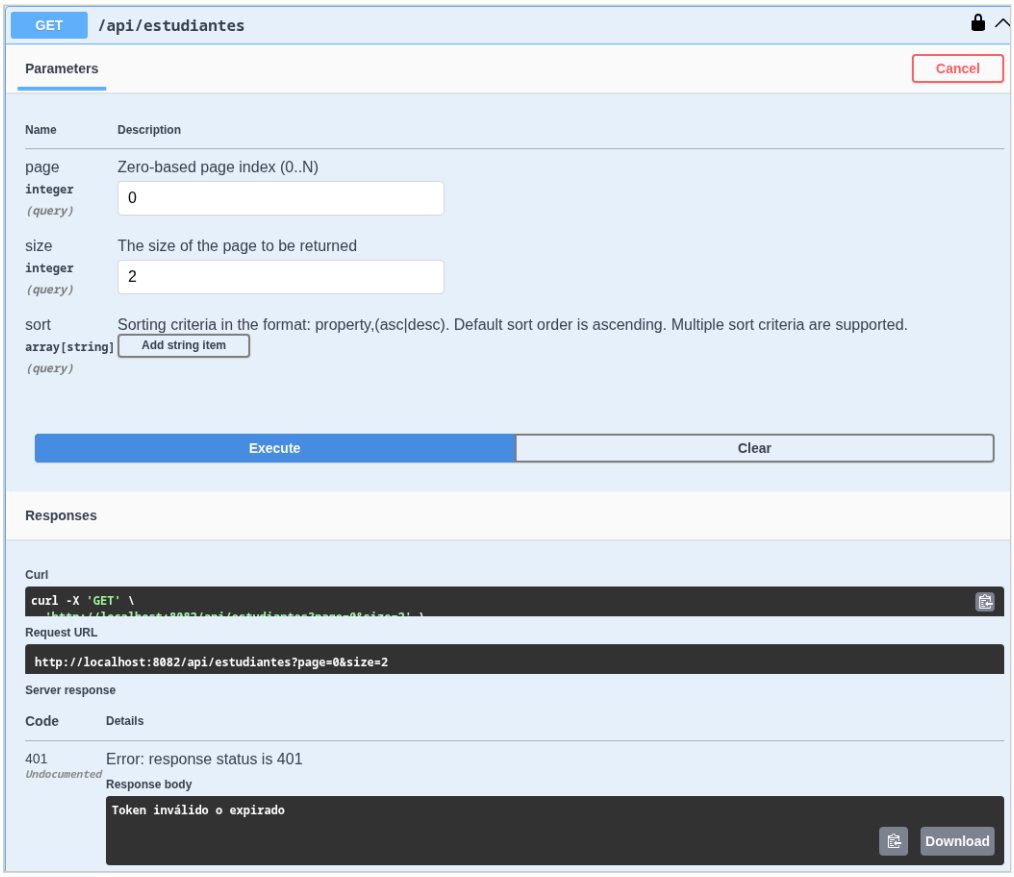

GET
/api/estudiantes
Parameters
Cancel
Name
Description
page
integer
(query)
Zero-based page index (0..N)
0
size
integer
(query)
The size of the page to be returned
2
sort
array[string]
(query)
Sorting criteria in the format: property,(asc|desc). Default sort order is ascending. Multiple sort criteria are supported.
Add string item
Execute
Clear
Responses
Curl
curl -X 'GET' \
Request URL
http://localhost:8082/api/estudiantes?page=0&size=2
Server response
Code
Details
200
Response body
{
"success": true,
"message": "OK",
"data": {
"content": [
{
"id": 1,
"nombre": "José Emilio",
"apellido": "Lozada",
"email": "estudiante0@demoacademico.edu",
"edad": 18
},
{
"id": 2,
"nombre": "Diana",
"apellido": "Martínez",
"email": "estudiante1@demoacademico.edu",
"edad": 19
}
],
"pageable": {
"pageNumber": 0,
"pageSize": 2,
Download
Response headers
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/json

Prueba 5 — GET /api/estudiantes sin autorizar



Prueba 6 — GET /api/estudiantes con token inválido

Se autorizó con el token alterado en sus últimos 4 caracteres, así que la firma no coincide:



Las capturas se tomaron con la solución corriendo en Docker Compose.

Pruebas adicionales

```
7. Validar token alterado en auth-service
GET http://localhost:8081/auth/validate?token=eyJhbGciOiJIUzI1NiJ9.e…
→ HTTP 401 ✔
{"valid": false, "message": "Token inválido o expirado"}

8. Token pegado con 'Bearer' de más (se normaliza)
GET http://localhost:8082/api/estudiantes/1
Authorization: Bearer Bearer eyJhbGciOiJIUzI1NiJ9.e…
→ HTTP 200 ✔
{"success": true, "message": "OK", "data": {"id": 1, "nombre": "José Emilio", "apellido": "Lozada",
"email": "estudiante0@demoacademico.edu", "edad": 18}}

9. Login de otro rol (docente) y validación
POST http://localhost:8081/auth/login
{"username": "docente", "password": "Docente2026*"}
→ HTTP 200 ✔
{"token": "eyJhbGciOiJIUzI1NiJ9.e…", "type": "Bearer"}

10. Validar token de docente
GET http://localhost:8081/auth/validate?token=eyJhbGciOiJIUzI1NiJ9.e…
→ HTTP 200 ✔
{"valid": true, "role": "DOCENTE", "username": "docente"}
```

El guion completo (pasos 1 a 6 más estos 4) se ejecutó dos veces: con los servicios corriendo localmente y en Docker Compose. En ambos casos el 10/10 pasos con el status esperado, con los mismos códigos HTTP. El paso 8 muestra que `TokenUtils.normalize` tolera un token pegado con un `Bearer` de más.